



Instituto Tecnológico Nacional de México

Instituto Tecnológico de Saltillo



Cómputo en la Nube

Miguel Salazar del Bosque

4.2 Demostracion la inyeccion SQL

Roberto Aram López Rodríguez

No. Control: 22050788

19-Marzo- 2026

INTRODUCCIÓN

Actualmente, muchas aplicaciones web manejan datos como usuarios y contraseñas, por lo que es importante cuidar la seguridad. Sin embargo, existen errores que pueden permitir que alguien acceda sin autorización, como la inyección SQL.

En esta práctica trabajé con un login sencillo para entender cómo funciona esta vulnerabilidad. Primero probé cómo se puede entrar sin contraseña y después modifiqué el código para hacerlo más seguro, lo que me ayudó a diferenciar entre un sistema vulnerable y uno protegido.

OBJETIVO

Comprender cómo funciona una inyección SQL en un sistema vulnerable y aplicar una solución para evitar este tipo de ataques mediante el uso de consultas seguras.

DESARROLLO

1. Preparación del entorno

Para comenzar, utilicé una máquina virtual con servidor web y base de datos. Se creó una base de datos llamada vulnerable_db junto con una tabla llamada usuarios, donde se almacenaron algunos registros de prueba como un usuario administrador.

Esto permitió tener un entorno básico para poder realizar las pruebas de inicio de sesión.

```
roberto@Debian-Server:~$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create database vulnerable_db;
Query OK, 1 row affected (0.012 sec)

MariaDB [(none)]> use vulnerable_db;
Database changed
MariaDB [vulnerable_db]> Create table usuarios(
  -> id int auto_increment primary key,
  -> nombre_usuario varchar(50),
  -> contrasena varchar(50)
  -> );
Query OK, 0 rows affected (0.051 sec)

MariaDB [vulnerable_db]> insert into usuarios(nombre_usuario, contrasena) values ('admin', '1234');
Query OK, 1 row affected (0.014 sec)

MariaDB [vulnerable_db]> insert into usuarios(nombre_usuario, contrasena) values ('user', 'password');
Query OK, 1 row affected (0.018 sec)

MariaDB [vulnerable_db]>
```

2. Creación de la aplicación vulnerable

Primero se creó el archivo login.php dentro del directorio del servidor web utilizando el editor nano desde la terminal.

Después, en este archivo se implementó un formulario de inicio de sesión junto con el código en PHP para validar el usuario y la contraseña.

Sin embargo, la consulta SQL se construye directamente con los datos que ingresa el usuario, sin ningún tipo de validación o protección, lo que hace que el sistema sea vulnerable a ataques de inyección SQL.

```
roberto@Debian-Server:~$ sudo nano /var/www/html/login.php
```

```
GNU nano 8.4 /var/www/html/login.php
<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);

$conn = new mysqli('localhost', 'root', 'root', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];

    // Consulta vulnerable
    $sql = "SELECT * FROM usuarios WHERE nombre_usuario='$usuario' AND contrase";
    $result = $conn->query($sql);

    if (!$result) {
        die("Error en la consulta: " . $conn->error);
    }

    if ($result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
GNU nano 8.4 /var/www/html/login.php
    if ($result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Login vulnerable</title>
</head>
<body>
    <form method="POST" action="login.php">
        <label>Usuario:</label>
        <input type="text" name="usuario"><br><br>

        <label>Contraseña:</label>
        <input type="password" name="contrasena"><br><br>

        <input type="submit" value="Ingresar">
    </form>
</body>
</html>

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

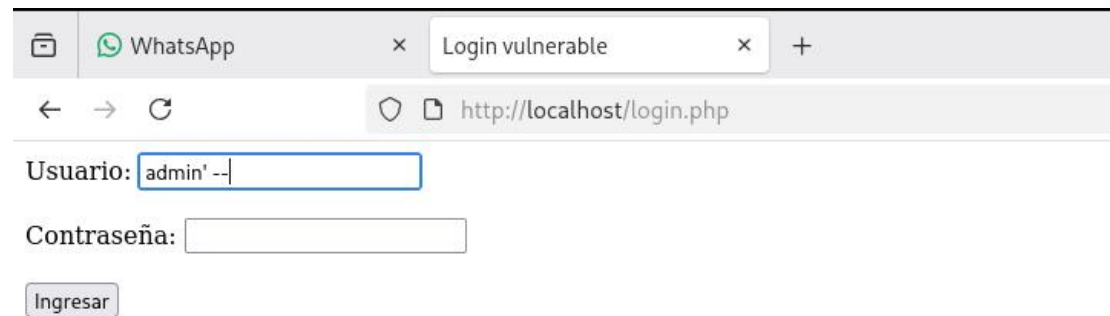
3. Prueba de inyección SQL

Se abrió la aplicación en el navegador y se realizó una prueba ingresando el siguiente valor en el campo usuario:

admin' --

y dejando la contraseña vacía.

Al enviar el formulario, el sistema permitió el acceso sin validar correctamente la contraseña, mostrando el mensaje **“Login exitoso”**. Esto demuestra que la aplicación tenía una vulnerabilidad de inyección SQL.



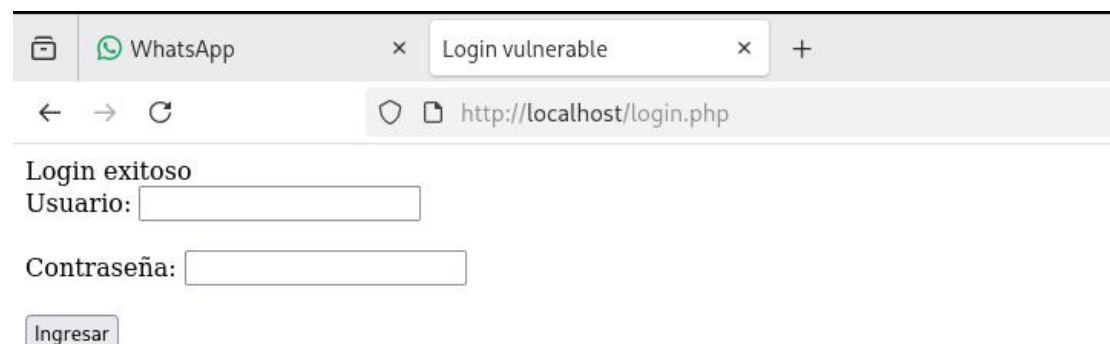
WhatsApp Login vulnerable

← → ↻ http://localhost/login.php

Usuario:

Contraseña:

Ingresar



WhatsApp Login vulnerable

← → ↻ http://localhost/login.php

Login exitoso

Usuario:

Contraseña:

Ingresar

4. Corrección de la vulnerabilidad

Para solucionar este problema, se modificó el código utilizando consultas preparadas. De esta forma, los datos que ingresa el usuario ya no se interpretan como parte de la consulta SQL, sino como valores separados.

Con este cambio, el sistema ya no permite que se altere la consulta con código malicioso.

```
GNU nano 8.4 /var/www/html/login.php
<?php
error_reporting(E_ALL);
ini_set('display_errors', 1);

$conn = new mysqli('localhost', 'root', 'root', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];

    // Consulta segura
    $stmt = $conn->prepare("SELECT * FROM usuarios WHERE nombre_usuario=? AND c");

    if (!$stmt) {
        die("Error en prepare: " . $conn->error);
    }

    $stmt->bind_param("ss", $usuario, $contrasena);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows > 0) {
        echo "Login exitoso";
    }
}

[ Read 43 lines ]
GNU nano 8.4 /var/www/html/login.php
if (!$stmt) {
    die("Error en prepare: " . $conn->error);
}

$stmt->bind_param("ss", $usuario, $contrasena);
$stmt->execute();
$result = $stmt->get_result();

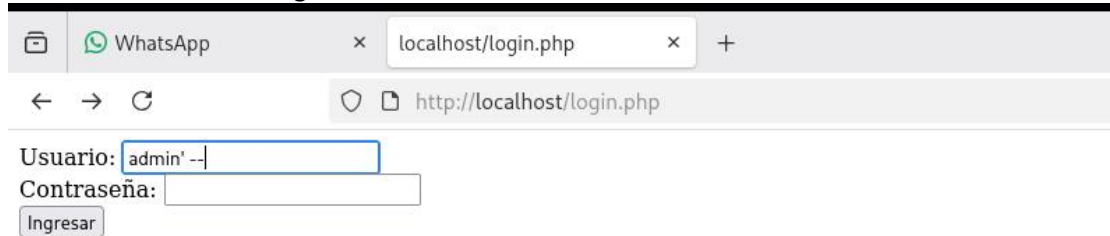
if ($result->num_rows > 0) {
    echo "Login exitoso";
} else {
    echo "Credenciales inválidas";
}
}
?>

<!DOCTYPE html>
<html>
<body>
<form method="POST">
    Usuario: <input type="text" name="usuario"><br>
    Contraseña: <input type="password" name="contrasena"><br>
    <input type="submit" value="Ingresar">
</form>
</body>
</html>
```

5. Verificación de la solución

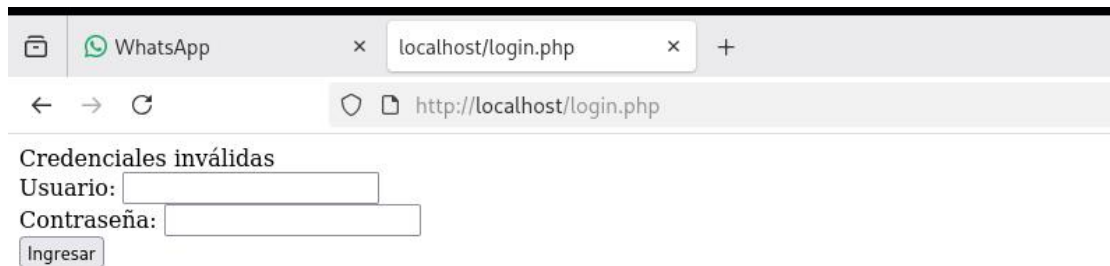
Finalmente, se repitió la misma prueba utilizando el mismo valor:
admin' --

Esta vez, el sistema mostró el mensaje **“Credenciales inválidas”**, lo que indica que la vulnerabilidad fue corregida correctamente.



A screenshot of a web browser window with two tabs: 'WhatsApp' and 'localhost/login.php'. The address bar shows 'http://localhost/login.php'. The login form contains the following fields and elements:

- Usuario:
- Contraseña:
- Ingresar button



A screenshot of a web browser window with two tabs: 'WhatsApp' and 'localhost/login.php'. The address bar shows 'http://localhost/login.php'. The login form displays an error message and contains the following fields and elements:

- Credenciales inválidas
- Usuario:
- Contraseña:
- Ingresar button

CONCLUSIÓN

En esta práctica pude darme cuenta de lo fácil que puede ser vulnerar un sistema cuando no se validan correctamente los datos que ingresan los usuarios. Me sorprendió ver que con algo tan simple como escribir un pequeño fragmento de código se puede acceder sin contraseña.

También entendí la importancia de aplicar buenas prácticas de programación, como el uso de consultas preparadas, ya que con un pequeño cambio se puede evitar completamente este tipo de problemas.

En general, esta actividad me ayudó a entender mejor la seguridad en aplicaciones web y la importancia de desarrollar sistemas más confiables.